Reinforcement learning
Assignment N.1

May 6, 2025

Bar, Alon
205476013

---

1 Theory

1.1 Markov chain

1. Draw the state transition diagram.

Ans:

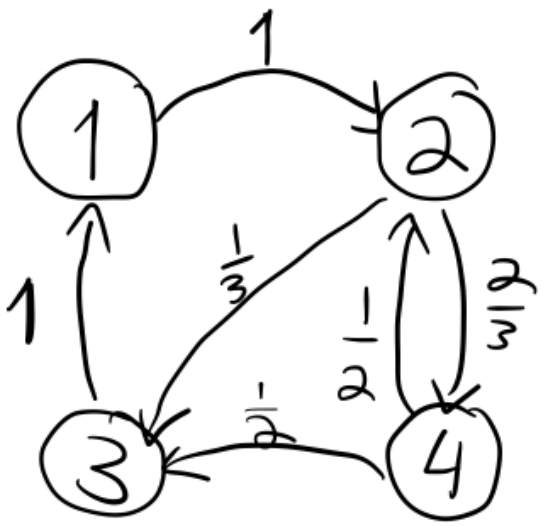


Figure 1: State transition diagram for question 1.1

2. State the communicating classes. Is the chain irreducible?

Ans: Since, for example, $4 \to 3 \to 1 \to 2 \to 4$, every state reaches every other. Hence there is a single communicating class S={1,2,3,4}, so the chain is irreducible.

3. Calculate di, the period of each state. Is the chain aperiodic?

Ans: $d_1 = gcd\{m \geq 1 : p_{1,1}^{(m)} > 0\}$

We could take trajectory (1, 2, 3, 1) where m = 3 and also (1, 2, 4, 3, 1) where m = 4 so the gcd is 1.

State i is a-periodic if $d_i = 1$

All states in the same class have the same period

So the chain is a-periodic.

4. If possible, find the invariant distribution.

Ans:

This is irreducible + aperiodic +finite state space Markov chain

According to the theorem, we have a unique Invariant distribution

find μ such that $\mu P = \mu$:

$$(\mu_1, \mu_2, \mu_3, \mu_4) \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & \frac{2}{3} \\ 1 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 \end{pmatrix} = (\mu_1, \mu_2, \mu_3, \mu_4)$$

1. $0 * \mu_1 + 0 * \mu_2 + 1 * \mu_3 + 0 * \mu_4 = \mu_1$
   $\mu_3 = \mu_1$
2. $1 * \mu 1 + 0 * \mu_2 + 0 * \mu_3 + \frac{1}{2} * \mu_4 = \mu_2$
   $\mu_1 + \frac{1}{2}\mu_4 = \mu_2$
3. $0 * \mu 1 + \frac{1}{3} * \mu_2 + 0 * \mu_3 + \frac{1}{2} * \mu_4 = \mu_3$
   $\frac{1}{3}\mu_2 + \frac{1}{2}\mu_4 = \mu_3$
4. $0 * \mu 1 + \frac{2}{3} * \mu_2 + 0 * \mu_3 + 0 * \mu_4 = \mu_4$
   $\frac{2}{3} * \mu_2 = \mu_4$

5. Combining equations (2) and (4) gives $\mu_1 = \frac{2}{3}\mu_2$
6. Combining equations (3) and (1) and (5) gives $\frac{1}{3}\mu_2 + \frac{1}{2}\mu_4 = \frac{2}{3}\mu_2 \rightarrow \mu_4 = \frac{2}{3}\mu_2 = \mu_1$
7. We know that $\sum_i \mu_i = 1 \rightarrow \frac{2}{3}\mu_2 + \mu_2 + \frac{2}{3}\mu_2 + \frac{2}{3}\mu_2 = 1 \rightarrow \mu_2 = \frac{1}{3}$

So $\mu = (\frac{2}{9}, \frac{1}{3}, \frac{2}{9}, \frac{2}{9})$

5. For every state $i \in S$, calculate $E[T_i]$, the expected return time to state i (hint: use the previous clause). Which states are recurrent? Classify each recurrent state as null or positive.

Ans:

The chain is a-periodic, irreducible and has finite state space $\rightarrow$ the chain is positive recurrent.

$\mu_i = \frac{1}{E[T_i]}$ $\frac{2}{9} = \frac{1}{E[T_{1,3,4}]} \rightarrow E[T_{1,3,4}] = 4.5$ $\frac{1}{3} = \frac{1}{E[T_2]} \rightarrow E[T_2] = 3$

6. Write a new transition matrix P' so that E[T1] = 3 and d1 = 3. Calculate $P^m_{1,1}$ as a function of m for that transition matrix, given that 1 is the initial state

Ans: $P' = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$

The only trajectory possible is (1, 2, 3, 1, 2, 3, 1, ...)

$d_1 = gcd\{m \geq 1 : p^{(m)}_{1,1} > 0\}$ $p^m_{1,1} = \begin{cases} 1, & m \bmod 3 = 0 \\ 0, & else \end{cases}$ $d_1 = 3$ $E[T_i] = 3$

1.2 Bidirectional MDP

1. Write a formal MDP for this problem (i.e., $\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}$ and s_0).

Ans:

$\mathcal{S} = \{0, 1, ..., 2k-1\}$

$\mathcal{A} = \{CW, CCW\}$

$$P(s'|s, CW) = \begin{cases} 1, s' = (s+1) \bmod 2k \\ 0, else \end{cases}$$

$$P(s'|s, CCW) = \begin{cases} 1, s' = (s-1) \bmod 2k \\ 0, else \end{cases}$$

$$\mathcal{R}(s, a) = \begin{cases} 1, & s = 0 \\ 0, & else \end{cases} \forall\, a$$

Finally, the return is discounted with factor $\gamma < 1$

$s_0 = k$

2. What is the optimal policy π^*?

Ans:

$$\pi^* = \begin{cases} CW, & s \in \{0, 1, ..., k\} \\ CCW, & else \end{cases}$$

3. After one iteration of the Value Iteration algorithm (where all states start with a value of 0), which states change their value after one iteration? what are the new values?

Ans:

$V_1(S) = max_{a \in A_1}\{r_1(s, a) + \gamma \sum_{s' \in S_0} p_1(s'|s, a) V_0(s')\}$

$V_0(S) = 0 \rightarrow V_1(S) = max_{a \in A_1}\{r_k(s, a)\}$

$$V_1(S) = \begin{cases} 1, & s = 0 \\ 0, & else \end{cases}$$

4. Which states change their value after two iterations? what are the new values?

Ans: $V_2(S) = max_{a \in A_2}\{r_2(s,a) + \gamma \sum_{s' \in S_1} p_2(s'|s,a) V_1(s')\}$
$V_2(s_0) = max_{a \in A_2}\{1+0\} = 1$ $V_2(s_{1,2k-1}) = max_{a \in A_2}\{0 + \gamma * 1 * 1, 0+0\} = \gamma$
$V_2(s_{else}) = max_{a \in A_2}\{0+0\} = 0$

5. For k = 2 (4 states), calculate $V^*(s)$ for every $s \in S$ (as a function of γ).

Ans: We will use Bellman equation:
$V^*(s) = max_{a\ inA}\{r(s,a) + \gamma \sum_{s' \in S} p(s'|s,a) V^*(s')\}$

1. $V^*(s_0) = max_{a\ in\{CW,CCW\}}\{1 + \gamma \sum_{s' \in S} V^*(s')\} = 1 + \gamma V^*(1) = 1 + \gamma V^*(s_3)$
2. $V^*(s_1) = V^*(s_3) = max_{a\ in\{CW,CCW\}}\{0 + \gamma \sum_{s' \in S} V^*(s')\} = 0 + \gamma V^*(s_0) = \gamma V^*(s_0)$
   By symmetry $V^(1) = V^(3)$, so we only need to track one of them.
3. $V^*(s_2) = max_{a\ in\{CW,CCW\}}\{0 + \gamma \sum_{s' \in S} V^*(s')\} = 0 + \gamma V^*(s_1) = \gamma^2 V^*(s_0)$
   Combining equations (1) and (2) gives
   $V^*(s_0) = 1 + \gamma^2 V^*(s_0) \rightarrow V^*(s_0) = \frac{1}{1-\gamma^2}$
   $V^*(s_1, s_3) = \frac{\gamma}{1-\gamma^2}$
   $V^*(s_2) = \frac{\gamma^2}{1-\gamma^2}$

$$V^*(s) = \begin{cases} \dfrac{1}{1-\gamma^2}, & s = 0, \\ \dfrac{\gamma}{1-\gamma^2}, & s \in \{1,3\}, \\ \dfrac{\gamma^2}{1-\gamma^2}, & s = 2. \end{cases}$$

1.3 Digit game

1. Build an MDP for this problem. (Hint: you may incorporate the random digit in the state.)

Ans:

$S = \{< m, d >: m \subseteq (0, 1, ..., N), d \in (0, 1, ..., 9)\}$

Final state $= < \emptyset, d >$ for every d

In any nonterminal (m, d), you choose one empty slot:

$A(s) = \{m :< m, d >= s\}$

Placing digit d into slot i yields

$R(s, a) = d * 10^a$ `where` $s =< m, d >$

If you take action $i \in m$ in (m, d), then

$$p(s'|s, a) = \begin{cases} 0.1, & m' = m \setminus \{a\} \text{ for each next digit } d \in 0, 1, ..., 9 \\ 0, & else \end{cases}$$

$\gamma = 1$

2. Show that the decision of the optimal policy depends only on the number of empty slots and the current random digit d_i.

Ans: We will prove this by backwards induction.

For 0, 1 digit left there is no meaning to the policy).

We will check the case for 2 digits left $s =< (a, b), d >: a, b \in (0, 1, ..., N), d \in (0, 1, ..., 9)$ where $a > b$

$V(s) = max(d * 10^a + \frac{1}{10} \sum_{d'=0}^{9} V(< (b), d' >), d * 10^b + \frac{1}{10} \sum_{d'=0}^{9} V(< (a), d' >)$

we will now see when $d * 10^a + \frac{1}{10} \sum_{d'=0}^{9} V(< (b), d' >) > d * 10^b + \frac{1}{10} \sum_{d'=0}^{9} V(< (a), d' >$

$V(< (a), d' >) = d' * 10^a)$, $V(< (b), d' >) = d' * 10^b)$ Only one choice left.

$d * 10^a + \frac{1}{10} \sum_{d'=0}^{9} d' * 10^b > d * 10^b + \frac{1}{10} \sum_{d'=0}^{9} d' * 10^a$

$d * 10^a + 4.5 * 10^b > d * 10^b + 4.5 * 10^a$

$d(10^a - 10^b) > 4.5(10^a - 10^b) \rightarrow d > 4.5$

so for $d \geq 5$ we will get that V will be chosen by the bigger digit - and we can see it only depends on the current random digit.

Let's assume that for k open spots the assumption is correct

$s =< m, d >: |m| = k, m \subseteq (0, 1, .., N), d \in (0, 1, ..., 9)$

The optimal value function in any state (m,d) with $|m| = k$ depends only on k and the digit d, not on the particular choice of which indices are in m.

We will test it for k+1

$s =< m, d >: |m| = k+1, m \subseteq (0, 1, .., N), d \in (0, 1, ..., 9)$

$V(s) = V(< m, d >) = max_{i \in m}\{d * 10^i + \frac{1}{10}\sum_{d'=0}^{9} V(< m', d' >)\}$

For each factor in the max we got two contributers - $d * 10^i$ which depends only on i which depends on the open spots in the state and the digit d, and the other side is $V(< m', d' >)$ which by the induction assumption depeneds only on the k open spots and the digit d'.

3. Compute the optimal policy for N = 2. (I.e the number has three digits.)

Ans: $V_1(< \emptyset, d >) = 0$ $V_1(< (a), d >) = d * 10^a \rightarrow \pi^* = a$ We don't have a choice

$V_2(< (0,1), d >) = max\{d*1+4.5*10, d*10+4.5*1\} = max\{d+45, 10d+4.5\}$
$V_2(< (0,2), d >) = max\{d*1+4.5*100, d*100+4.5*1\} = max\{d+450, 100d+4.5\}$
$V_2(< (1,2), d >) = max\{d*10+4.5*100, d*100+4.5*10\} = max\{10d+450, 100d+45\}$

So as we proven before when $d \geq 5$ chose the higher index

$$\pi^*(< a, b >, d) = \begin{cases} a, & d \geq 5 \\ b, & else \end{cases} \text{ where } a > b$$

$V_3(< (0,1,2), d >) = max\{d*1+\frac{1}{10}\sum_{d'=0}^{9} V_2(< (1,2), d' >), d*10+\frac{1}{10}\sum_{d'=0}^{9} V_2(< (0,2), d' >), d*100+\frac{1}{10}\sum_{d'=0}^{9} V_2(< (0,1), d' >)\}$

$V_3(< (0,1,2), d >) = max\{d+607.5, 10d+578.25, 100d+60.75\}$

$$\pi^*(< (0,1,2), d >= \begin{cases} 0, & d < 4 \\ 1, & d \in \{4,5\} \\ 2, & d > 5 \end{cases}$$

1.4 MDP & Operator

1. Show that T is a γ-contracting with respect to the max-norm

Ans:

$TV(s) = \frac{1}{|A|}\sum_{a \in A}[r(s,a) + \gamma \sum_{s' \in S} p(s'|s,a)V(s')]$

$|TV_1(s) - TV_2(s)| = |\frac{1}{|A|}\sum_{a\in A}[r(s,a) + \gamma\sum_{s'\in S} p(s'|s,a)V_1(s')] - \frac{1}{|A|}\sum_{a\in A}[r(s,a) + \gamma\sum_{s'\in S} p(s'|s,a)V_2(s')]|$

$= |\frac{1}{|A|}[\sum_{a\in A}[r(s,a) - r(s,a)] + \sum_{a\in A}\gamma\sum_{s'\in S} p(s'|s,a)V_1(s') - p(s'|s,a)V_2(s')]]|$

$= |\frac{1}{|A|}\sum_{a\in A}\gamma\sum_{s'\in S} p(s'|s,a)V_1(s') - p(s'|s,a)V_2(s')]|$

$= |\frac{1}{|A|}\sum_{a\in A}\gamma\sum_{s'\in S} p(s'|s,a)(V_1(s') - V_2(s')]| \leq \frac{1}{|A|}\sum_{a\in A}\gamma\sum_{s'\in S} p(s'|s,a)|(V_1(s') - V_2(s')|]$

Triangle inequality

$\leq \gamma|V_1(s) - V_2(s)|$ Because $(\frac{1}{|A|}\sum_a\sum_{s'} p(s'|s,a) = 1\Big)$

$\leq \gamma\|V_1(s') - V_2(s')\|_\infty$ Because $(\max_{s'}|V_1(s') - V_2(s')| = \|V_1 - V_2\|_\infty\Big)$

Taking the supremum over s on the left gives

$$\|TV_1 - TV_2\|_\infty = \sup_s\big|(TV_1)(s) - (TV_2)(s)\big| \;\leq\; \gamma\,\|V_1 - V_2\|_\infty,$$

which shows T is a γ-contraction in the max-norm.

2 Programming

2.1 Value Iteration

1. Implement the relevant code in the attached value_iter.py code. The relevant locations are marked with REPLACE THIS LINE WITH YOUR CODE comment.

Ans:

```
For example, state 0 is the initial state, and the transition information for s=0, a=0 is
P[0][0] = [(0.3333333333333333, 0, 0.0), (0.3333333333333333, 0, 0.0), (0.3333333333333333, 4, 0.0)]

As another example, state 5 corresponds to a hole in the ice, in which all actions lead to the same state with probability 1 and reward 0.
P[5][0] = [(1.0, 5, 0)]
P[5][1] = [(1.0, 5, 0)]
P[5][2] = [(1.0, 5, 0)]
P[5][3] = [(1.0, 5, 0)]

Iteration | max|V-Vprev| | # chg actions | V[0]
----------+--------------+---------------+---------
   0      | 0.33333      |  N/A          | 0.000
   1      | 0.10556      |   1           | 0.000
   2      | 0.06685      |   1           | 0.000
   3      | 0.06351      |   2           | 0.000
   4      | 0.04357      |   1           | 0.000
   5      | 0.03821      |   4           | 0.003
   6      | 0.02857      |   2           | 0.008
   7      | 0.02437      |   1           | 0.014
   8      | 0.01952      |   1           | 0.021
   9      | 0.01624      |   0           | 0.028
  10      | 0.01384      |   0           | 0.036
  11      | 0.01173      |   0           | 0.044
  12      | 0.01047      |   1           | 0.052
  13      | 0.00948      |   0           | 0.060
  14      | 0.00852      |   1           | 0.068
  15      | 0.00782      |   0           | 0.075
  16      | 0.00733      |   0           | 0.083
  17      | 0.00694      |   0           | 0.090
  18      | 0.00656      |   0           | 0.096
  19      | 0.00618      |   0           | 0.102
```

Figure 2: Python output for value_iter.py

2. plot of the optimal policy at each iteration

Ans:

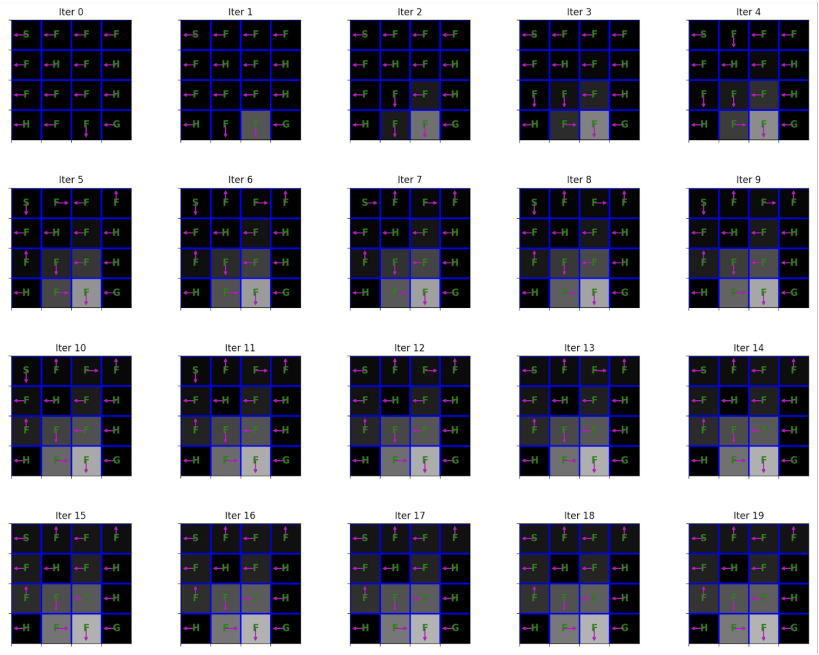


Figure 3: Optimal policy at each iteration

3. plot of each state value as a function of iterations (different curve for each value, same figure)

Ans:

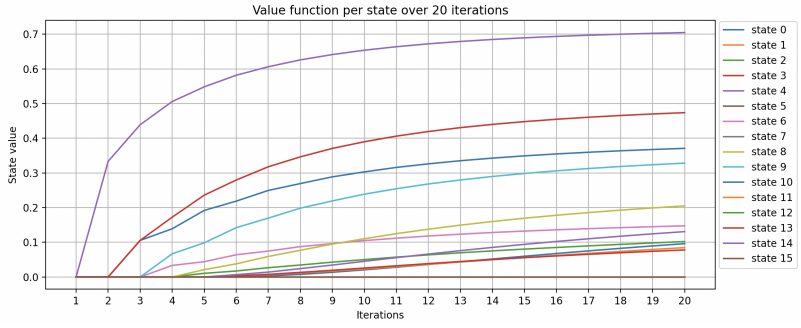


Figure 4: Each state value as a function of iterations for value iteration

2.2 Policy iteration

1. Implement the function called compute_vpi

Ans: Done in policy_iter.py

2. Implement the function called compute_qpi

Ans: Done in policy_iter.py

3. Using the functions you implemented in parts (1) and (2) implement the function called policy_iteration which implements policy iteration.

Ans:

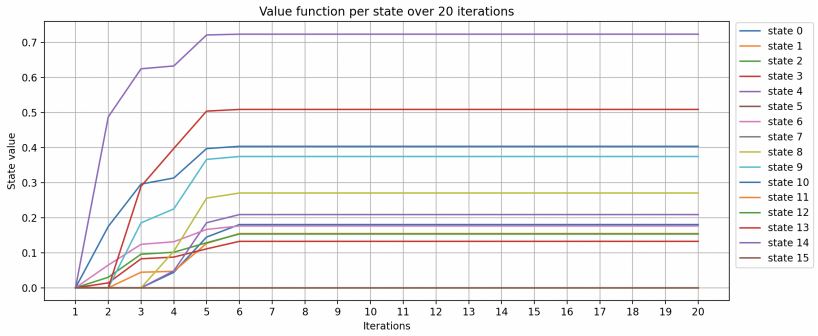


Figure 5: Each state value as a function of iterations for policy iteration

```
As another example, state 5 corresponds to a hole in the ice, in which all actions lead to the same state with probability 1 and reward 0.
P[5][0] = [(1.0, 5, 0)]
P[5][1] = [(1.0, 5, 0)]
P[5][2] = [(1.0, 5, 0)]
P[5][3] = [(1.0, 5, 0)]

Policy Value:  [ 1.879e-17  2.199e-02  6.945e-02  5.998e-02  2.176e-17  0.000e+00
  8.988e-02  2.002e-17  2.816e-17  1.541e-01  2.144e-01 -2.483e-16
  3.901e-17  2.721e-01  5.872e-01  0.000e+00]
Policy Action Value:  [ 1.879e-17  2.199e-02  6.945e-02  5.998e-02  2.176e-17  0.000e+00
  8.988e-02  2.002e-17  2.816e-17  1.541e-01  2.144e-01 -2.483e-16
  3.901e-17  2.721e-01  5.872e-01  0.000e+00]
Iteration | # chg actions | V[0]
----------+---------------+---------
   0      |      1        | 0.00000
   1      |      6        | 0.00000
   2      |      6        | 0.00000
   3      |      6        | 0.04421
   4      |      2        | 0.14475
   5      |      0        | 0.18047
   6      |      0        | 0.18047
   7      |      0        | 0.18047
   8      |      0        | 0.18047
   9      |      0        | 0.18047
  10      |      0        | 0.18047
  11      |      0        | 0.18047
  12      |      0        | 0.18047
  13      |      0        | 0.18047
  14      |      0        | 0.18047
  15      |      0        | 0.18047
  16      |      0        | 0.18047
  17      |      0        | 0.18047
  18      |      0        | 0.18047
  19      |      0        | 0.18047
```

Figure 6: Python output for policy_iter.py